赛题 19：动态算子图编译与并行调度

参数化块任务图、源码浮点语义与独立验证

本地项目：`19-dynamic-graph`

2026 年 10 月 11 日

摘要

本项目将受支持的实际 C 源码降低为包含可执行字节码和符号循环、访存表达式的 Metadata，面对动态矩阵规模执行 Cholesky 分解。不能证明可变换的源码由通用 VM 保持源码顺序；能够证明完整运算图的左看、官网右看及连续分块形式，使用参数化 POTRF/TRSM/UPDATE 任务 DAG。优化保持每个输出位置的乘加归约次序，禁止隐式 FMA；显式 FMA 作为独立源码语义保留。

本轮已完成任务 DAG、因子块打包、闭式依赖后继、独立累加器与 CPU 检测 SIMD，并扩大源码前端、helper 顺序求值、计数型 while 证明和整数转换测试。一般稠密 SPD 的最大规模 $N = 10000$ 已经运行原始 C 参考程序，并对完整 800,000,008 字节输出逐字节比较通过。报告区分一般稠密工作负载与对角/带状结构专门化，区分计算、任务图、文件 IO 与外部进程时间，并测量 4/8/16 线程伸缩和实际 RAM 峰值。保守 row 默认与显式 memory 配置各 7 项 CTest 通过；owned 输入复用令一般 dense10000 峰值约 2.02GB 降至 1.22GB，保留吞吐负结果。所有成绩是公开本地测试证据，不代表官方隐藏评测分数。

1 项目目标与题面契约

官网题目要求从朴素或分块 Cholesky 的 C 源生成算子符号依赖 Metadata，再由 Runtime 处理动态 Shape，$3 \le N \le 10000$。题面分数由 Metadata 大小 15 分和计算耗时 85 分构成，IO 不计入计算性能，结果完全一致为硬门槛 [1]。本项目因此同时报告文件字节数、计算时间与严格结果比较；没有正式评分函数、评测 CPU 或官方数据包时，不推算最终分数。

官网源码图片采用右看更新：先对角开方，再计算本列下方因子，随后将乘积累加到后续下三角工作区。图片没有给出 `div`/`madd` 定义或输出初始化。明确的本地契约为：$\mathrm{div}(x, y) = x/y$，非融合 $\mathrm{madd}(x, y, z) = x \times y + z$，$L$ 初始全为正零，上三角保留正零；另设显式 `fma` 源码验证不同语义。自设分块 C 不是官方分块源码。

输入、输出均采用小端 `uint32` 矩阵数 M，随后每矩阵一个 `uint32` 维度 N 与行优先 N^2 个 FP64 值。单矩阵文件长度为 $8 + 8N^2$ 字节，多矩阵长度为 $4 + \sum_z (4 + 8N_z^2)$。本地额外限制 $0 \le M \le 100000$，允许空批次；这是资源保护契约，官网未给出该上限。输入为有限 SPD 矩阵，默认 IEEE 浮点环境，舍入到最近值；不改变 FTZ/DAZ 或舍入模式。符号零属于严格比较内容。

2 重要工作及其对本项目的影响

2.1 LLVM 与多面体分析

Polly 在 LLVM IR 上识别可建模区域，构建计算与访存模型，变换后重新生成 IR；其文档还说明规范化、内联及 LLVM 优化管线位置会影响可分析性 [2]。这支持本项目先建立符号循环与访存表达式、再进行可证明变换的分层设计。目前实际前端是 Clang AST 降低，并导出 LLVM IR 供审计，没有集成 Polly 或实现任意 C 的多面体编译器。整数地址交换、等价循环边界的规范化只用于整数符号证明，浮点归约表达式不作代数重排。

2.2 任务化线性代数

Buttari 等的分块线性代数工作将 Cholesky 拆成对角分解、三角求解和尾部更新任务，通过依赖 DAG 释放并行度，并强调数据分块与任务调度共同影响局部性 [3]。LAPACK 的 `DPOTRF` 采用阻塞形式与 Level-3 BLAS[4]。本项目采用相似的任务分解，但保留官网的独立 A 与 L 累加工作区，以及逐位置正向累加顺序；直接替换为普通 BLAS 不能自动保证给定源码的逐位真值。

PaRSEC 以符号任务流表示依赖，其 JDF 可以在运行时局部计算依赖而不展开整个 DAG[5]。本项目先把堆分配后继列表改为闭式后继公式，再实现每目标块一个当前 panel 状态，将研究候选的常驻状态从 $O(t^3)$ 降至 $O(t^2)$，其中 $t = \lceil N/B \rceil$。这只减少状态，不减少必须执行的浮点任务；一般符号图、工作窃取和局部性调度仍有工作余量。StarPU 的 codelet、数据句柄和读写访问模式能帮助运行时自动建立依赖 [6]；本项目借鉴其依赖与 kernel 分离的思路，未引入 StarPU，也未测试 GPU 执行。

2.3 确定性 FP64 与源码一致性

LLVM 的浮点语义区分默认环境、严格环境及 fast-math 标记；`reassoc` 和 `contract` 允许的变换会影响归约与融合 [7]。Clang 文档说明，仅指定`-fno-fast-math` 并不能取代显式`-ffp-contract=off`[8]。因此参考 C 和 Runtime 均同时关闭 fast-math 与隐式收缩。

Ahrens、Demmel 和 Nguyen 通过可复现累加器使求和结果与求和顺序无关 [9]。本项目的要求更具体：与给定普通逐项 C 运算的位模式相同。由此推得，改用更准确或顺序无关的累加器也不能自动满足源码真值；本项目保留每个位置的加法链，而只并行不同位置。显式 FMA 源使用 FMA 语义，非融合源始终先乘后加。

3 算法与实现

3.1 从实际源到可执行 Metadata

编译链为实际 C 源、Clang JSON AST、类型与声明身份、栈字节码及符号访存 IR、受信 Metadata、动态 Shape Runtime。变量名、函数名和源 hash 不参与算法分派。改变实际乘加算式会改变程序，并使不满足证明条件的源码回退到 VM；参考结果由另一条路径直接编译原始 C 源产生。

支持签名为 `int/void f(double *A,double *L,int n[,int block])`，输入参数亦可为 `const double*`。支持有符号 32 位 `int`、`double`、直接数组下标、`for/while/if`、条件与短路表达式、基本算术、`sqrt/fabs/fmin/fmax/fma`，以及纯标量 helper。新增 helper 可以包含按声明顺序初始化的不可变 SSA 局部值，最终一个 return；有副作用、控制流或未初始化读取的 helper 拒绝。

VM 中的整数算术结果和 double 转 int 均进行 int32 范围检查，零规范为正零。该处理修复了 `trunc(-0.01)` 产生负零、而 C 转换到 int 后再提升应为正零的差异；整数除法、余数、一元负号也有相应回归。未定义的整数溢出或除法不作为可执行 C 语义接受。

Metadata 保留完整可执行程序，以及循环、父循环、读写数组和参数化下标。重复符号表达式编码为共享表达式 DAG，可无损还原；可选 expanded 模式用于审计。来源与 Metadata 的 SHA 记录在分析 JSON 中，可按需内嵌来源信息。程序文件与矩阵维度无关；运行时绑定 N 及源码的可选 `block` 参数。物理调度块 B 是另一个参数，不要求等于源码分块参数。

3.2 参数化块任务图

令 $t = \lceil N/B \rceil$，对 $0 \le k < t$ 定义对角任务 $P(k)$，对 $k < m < t$ 定义三角求解 $T(m,k)$，对 $k < h \le m < t$ 定义尾部更新 $U(m,h,k)$。依赖为

$$
\begin{aligned}
P(k) &\leftarrow U(k,k,k-1),\\
T(m,k) &\leftarrow P(k),\ U(m,k,k-1),\\
U(m,h,k) &\leftarrow T(m,k),\ T(h,k),\ U(m,h,k-1).
\end{aligned}
$$

域外依赖省略；对角更新中的同一 T 只计一次。任务总数为

$$
Q(t) = \sum_{q=1}^{t} \frac{q(q+1)}{2} = \frac{t(t+1)(t+2)}{6}.
$$

每个任务使用 8 字节：三个 16 位块坐标、任务种类和未完成依赖数。节点后继由闭式索引计算；不为每个任务分配 `vector` 边列表。就绪优先队列偏向靠近对角的目标列，工作线程持续存在到整个矩阵完成。更新完成后才释放后继；每个输出块对不同 k 的更新形成严格链。

任务 P 处理对角块的原始递增列次序，T 处理对应行块的原始除法和块内累加，U 按递增源列更新尾部块。同一输出位置经历的乘法、加法、减法、除法和开方次序不变；不同位置的独立运算可异步。循环尾部统一以真实 N 截断，支持奇数维度、$B > N$ 与非整块。

3.3 缓存、独立累加器与 CPU SIMD

已经完成的因子块按块内列打包，提供连续的右侧因子访问。新增缓存为

$$
8B^2 \frac{t(t+1)}{2} \text{ 字节} \approx 4N^2,
$$

输入和输出合计为 $16N^2$ 字节。微内核同时维护 4、8 或 16 个不同输出位置的累加器，不拆分单个位置的归约树。因子数据不可变，更新块之间的写区域独立。

AVX2 路径放在单独编译单元，通用 CPU 检测器使用基础指令集，检测 AVX2 与 FMA 特性后才调用。该方法遵循 GCC 关于 CPU 检测和架构专用代码分离的说明 [10]。非融合路径使用分别的向量乘法和加法；只有显式 FMA 源码使用 `fmadd`。`--simd portable` 强制回退。本报告不把跨 CPU、不同数学库或不同浮点环境的逐位一致性作为已验证结论。

3.4 IO 和失败处理

Runtime 逐矩阵读取、计算、写出，内存中不保留整批 M 个矩阵。检查维度、长度、尾部数据、Metadata 参数和跳转；错误结果不发布，临时 `.partial` 文件清理。为避免手工设置极小 B 展开不可负担的 DAG，最多物化 1600 万个任务；$N = 10000, B = 32$ 处于此限额内。

4 验证方法与已完成成果

4.1 独立参考与数据集

参考工具直接包含并编译实际 C 源，使用 GCC15.2、`-O2`、`-fno-fast-math` 与`-ffp-contract=off`。它不使用 Runtime、AST VM 或复制 Runtime 的算法产生真值。比较覆盖头部、全部下三角与上三角、所有符号零；不以残差或容差替代逐字节比较。

一般稠密 SPD 用对称、严格对角占优矩阵构造：对角 $N+2$，非对角为确定性哈希产生的正负小数。FP 压力数据包括 *DCD* 混合尺度、真实 subnormal 元素、约 2^{900} 量级大数、接近秩一 SPD 和正负零。自设源码覆盖左看、官网右看、动态连续分块、显式 FMA、变量与循环形式变化、SSA helper、改变算式以及通用 while/if 程序。

表 1: 本轮里程碑；各批证据独立保存，不合并成官方测试数量

批次	数值对照	性能执行	证据范围
块 DAG 与打包	60	240	4 种源、奇数/尾块、$N = 256 \dots 2048$，5 轮
闭式图与大规模	96	62	真实 subnormal 等，$N = 4096, 10000$ 原始 C 真值
8/16 独立累加器	128	180	4 种源 FP 压力，3 个规模、5 轮轮换
扩展类型前端	188	已计入对照	13 种源；整数转换、负号、除法/余数
CPU 检测 SIMD	128	112	portable/AVX2、4 种源、3 个稠密规模
对角因子缓存	128	176	原始 C 严格比对、5 轮 A/B 与最大规模
Metadata 紧凑编码	24	不作性能评分	4 编码、3 种源、VM 和块 DAG
对齐与就绪队列	136	264	奇数尾块、64/128 列 mask 边界、10k 重复
两行复用与线程唤醒	128	252	逐位置 k 链保持；5 轮或最大 3 轮

批次	数值对照	性能执行	证据范围
自动物理块策略	16	222	4 源、127/128 与 1024/1025 切换、手动覆盖
四行向量复用	128	168	4 源 FP 压力、大规模三轮及小规模负结果
线程相关块成本	0	200	同一候选、10 个稠密规模、1/4 线程
helper 顺序与证明	49	不作性能评分	嵌套/prototype/线性 SSA；另 17 项拒绝
带状与新稠密族	380	46	符号零、下溢、带宽边界、真实 10k C
八行实验	128	128	强制 dense；保留性能负结果
线程伸缩与 RAM	0	48	4/8/16 线程；普通 10k 及 M3 流式
线程与块几何	0	81	同一候选；B64/96/128/192，三轮
计数型 while 证明	63	不作性能评分	6 种源；分块 1/7/32/100；改算式回退
线程 Shape 分派	38	不作性能评分	4 源、1536/1537 及混合 Shape 边界
P/T 独立位置 SIMD	128	200	4 源 FP 压力；B32 负结果与 B64/96 收益
完整线程策略	144	116	自动 B、1/4/8/16、M3 与带状分别记录
有界解锁通知	208	156	相同运算与 B；16 线程负结果保留
整数比较选择 helper	105	不作性能评分	分块 for/while、FMA；另 12 项拒绝
缓存初始化与解锁通知	208	66	NaN poison、不清零；首次触页成本计入
连续列块工作区	209	94	正零、尾块、1/4/16；导出计入 compute
混合块布局与并行导出	418	188	四构建轮换；保持负值与共享长尾
惰性当前 panel 任务图	418	104	heap/bucket；另 30 项异常退出
等待计数与行前缀	418	156	自动与 B32；另 30 项异常退出
单 worker 闭式遍历	642	76	基础 418+ 单 worker224；另 30 失败
同源向量扫描	545	60	另 288 非有限拒绝、308880 位/保护页单测
输入 owned 复用	518	80	另 36 失败、API 所有权/const 验证
auto 复用与最终配置	854	74	另 288 拒绝 +42 失败；脚本失败/恢复保留

4.2 第一批缓存与调度收益

表2是同批轮换 A/B 的 5 次中位数，旧基线为持续线程池、每列两阶段屏障，新版本为块 DAG、打包与 4 独立累加器，均为普通稠密 SPD。单线程的明显提升表明缓存局部性和独立累加器贡献很大；不能把全部收益归因于多线程调度。$N = 2048$ 时新引擎单线程相对旧单线程约 12.53 倍，自己的 4 线程相对单线程约 3.82 倍；这两个比例代表不同因素。本批 $B = 32$ 优于 64、96、128，后续 ISA 优化仍重新搜索块大小。

表 2: 块 DAG 第一批，$B = 32$；单位 ms，计算包含分配与调度

N	旧 1 线程	新 1 线程	旧 4 线程	新 4 线程	同 4 线程加速比
256	5.334	1.274	5.962	0.916	6.51
512	98.964	12.283	32.755	3.975	8.24
1024	780.862	75.320	175.843	20.558	8.55
2048	6461.530	515.697	1286.530	135.039	9.53

4.3 闭式图的收益和限制

$N = 4096, B = 32$ 有 357,760 个任务。闭式后继把图构造约 55ms 降至约 6–8ms，静态任务元数据为 2,863,104 字节；总体计算改善为几个百分点，并非数量级。$N = 10000, B = 32$ 有 5,159,805 个任务，静态任务元数据 41,280,944 字节，旧版本图构造约 918ms，新版本约 60ms。就绪队列及其他容器占用另计；这些数字不是题目评分中的磁盘 Metadata 大小。

一般稠密 $N = 10000$ 的原始 C 参考单次为 504.670 秒，输入生成 34.767 秒。第二批 8 种配置均与其完整 800,000,008 字节输出相同，SHA256 为

```
4015abfccfe5851a9c959939ff61ee44acf0290fcbf6192cdfd59a3a217eef13
```

紧凑图、4 独立累加器、$B = 32$ 的 1 线程计算 49.174 秒，4 线程 13.743 秒；这是每配置一次的实测，不称为稳定中位数。$B = 64$ 的 4 线程 14.982 秒而旧版本 13.195 秒，显示大规模单次观测与并发负载有波动，不能仅以图构造减少推断所有配置总时间必然下降。

4.4 独立累加器与 SIMD

独立累加器批次中，$N = 2048, B = 32$ 的 1 线程 4/8/16 路中位分别 569.888、415.119、379.598ms；4 线程分别 145.635、119.390、109.237ms。较小 $N = 512/1024$ 的 4 线程 8 路略优于 16 路，选择必须考虑规模和调度成本。

CPU SIMD 批次已通过 128 组严格 FP 压力对照及 112 次稠密执行。$N = 2048, B = 32$ 三轮 4 线程中位为 portable 95.707ms、AVX2 65.599ms，约降低 31.5%；$B = 64/96/128$ 对应 AVX2 为 85.364/115.572/147.139ms。一般稠密 $N = 10000, B = 64$ 的单次 1 线程 portable/AVX2 为 30.578/14.523 秒，4 线程为 8.710/4.192 秒，完整 C 参考字节相同。该次 4 线程读入约 209ms、写出约 609ms，均另外计时。

4.5 对角因子缓存与紧凑 Metadata

第二次缓存优化让对角分解和行块求解直接使用已打包的对角因子，并在算出因子时立即写入缓存，取消独立打包扫描。$N = 2048, B = 64$ 的同批 5 轮 4 线程中位从 86.592 降至 36.443ms；$N = 4096$ 从 452.281 降至 227.748ms。每个位置的浮点链未改变，128 组 FP 压力与 176 次性能执行全部通过原始 C 参考。

最大规模单次结果并未显示相同收益：$N = 10000, B = 64$ 的 4 线程从 3.074 变为 3.312 秒，1 线程 13.167/13.175 秒基本持平。本批不把对角缓存概括为所有规模加速，后续独立轮换批次比

较缓存对齐和就绪队列，并保留运行区间及并发负载限制。

符号表达式共享 DAG、来源信息转移到分析 JSON、无参数字节码省去冗余零操作数，均保持可执行程序和展开符号图完全相同。左看、官网右看和自设分块的 Metadata 分别从同语义 expanded 编码 2680/3280/4445 字节降到 2250/2655/3739 字节，减少 16.0%/19.1%/15.9%。三种源的四种编码均通过 VM 与块 DAG 的独立 C 参考，共 24 次对照；这只是原始文件大小，不代入未知官方评分公式。

4.6 就绪队列、两行复用与自动物理块

分桶就绪队列按目标列分组，使用位掩码定位非空列，并优先推进对角任务；它与优先堆使用同一依赖图。136 组 FP/边界对照包含物理 $B = 1$ 跨 64/128 列 mask 边界，264 次稠密执行全部通过。$N = 4096, B = 64$ 的 4 线程对齐 heap/bucket 中位 233.590/234.562ms，最大规模三轮 4153.250/4092.860ms 且区间重叠。因此 heap 保持默认，bucket 作为实验选项；未声称分桶稳定加速。64 字节因子缓存对齐本批在 $N = 4096, B = 64$ 从 254.362 改善到 233.590ms。

两行向量内核让两个独立输出行共享右侧因子向量，保持各自递增 k 归约。它仍使用八个向量累加器，每个同时覆盖四个独立输出位置。选择通知只在退休释放一个或多个任务时唤醒 worker，完成与异常时唤醒全部，避免没有新增可执行工作的通知。128 组压力和 252 次稠密执行严格通过。表3为本批固定 $B = 64$ 的 4 线程中位，不和前批最好值混算。最大规模最终候选区间 2977.990–

表 3: 两行复用与选择通知，同批轮换；计算 ms

N	重复数	原单行	两行	两行加选择通知
2048	5	40.729	35.150	32.779
4096	5	239.099	221.794	199.052
10000	3	4059.180	3173.570	2984.180

3663.950ms。同批 1 线程只执行一次，原单行/两行/两行加通知为 14.555/11.394/12.047 秒；通知候选单次串行负观测保留，不用三轮 4 线程收益推出所有配置必然改善。

自动物理块在 $N \le 1024$ 选择 32、较大 N 选择 64，用户可显式覆盖；这是根据本地成本的任务几何策略，不影响数值结果或根据 Shape 查表结果。四种源共 16 次混合 Shape 对照检查 127/128 执行器切换与 1024/1025 物理块切换，并在 portable/AVX2、1/4 线程下保持原始 C 字节。222 次稠密执行同时比较自动策略和显式 $B = 32/64$。最大规模自动 4 线程三轮中位 3231.590ms，显式 64 为 3376.470ms、显式 32 为 5005.800ms；自动与 64 的差异来自观测波动，两者运行相同物理任务几何。有效块大小最小值/最大值与最后矩阵值由 Runtime 另行记录，混合 Shape 不以请求参数冒充实际 B。

4.7 helper 求值、四行复用与线程相关成本策略

helper 的重复 SSA 局部变量曾通过表达式代入展开，可能指数增长；未使用的参数或局部值也可能漏掉本应执行的整数检查。新前端对这类调用按参数、声明、return 的顺序降低为临时槽，所有参数先压栈，再写 callee 槽，因此同一 helper 嵌套不会覆盖外层待用参数。Clang 声明链中

的前向 prototype 归并到真实定义身份。20/40 层重复 SSA 分别降低为 146/266 条指令，线性而非指数增长；资源上限为 4096 槽、100,000 指令，并限制符号表达式展开。

原始 C 的 49 次整文件对照涵盖重复 SSA、同函数嵌套、多参数嵌套、条件 return、前向 prototype 和带未使用 double 值的 Cholesky。17 项拒绝检查覆盖未使用整数参数/局部值的溢出、真正递归、非有限常量、float/unsigned/wide helper 返回类型及不符 double 契约的数学函数签名；其中类似 Cholesky 但携带未使用整数检查的源码，在 auto、stage、tiled 及 VM 请求下均拒绝。优化证明保守排除有未使用求值、int 局部值或一般 int 参数运算的 helper；不能只比较返回表达式后忽略其余求值。下一小节所述纯整数比较选择器是明确的无溢出例外。

新增计数型 while 证明把相邻的单变量初始化声明、while 条件和末尾递增，仅在证明视图中规范化为 for。实际字节码和循环审计仍保留 while 控制流；全量运算、循环域、递增步长和访存证明仍必须通过。63 次原 C 比较覆盖左看、右看、显式 FMA、分块 while，源码 block 为 1/7/32/100 以及奇数/尾块；改变步长或算式的源码保持 VM。该批实测 20.073 秒，不宣称一般 while 自动并行。

4.8 分块边界的纯整数选择 helper

一般 int helper 保守留 VM 会让常见的 `min(panel+block,n)` 分块源失去任务 DAG。新增的例外严格限制为两个 int 参数、无局部声明、一个 return、直接比较两个参数并直接返回其中一个；实际传入表达式的类型也必须为 int。helper 自身的比较与选择不会发生 int32 算术溢出，调用参数的运算仍保留在完整循环/访存证明中。没有放开一般整数 helper 或丢弃其额外求值。

105 次原 C 比较验证 min 边界的分块 for、显式 FMA 与分块 while，源码 block 为 1/7/32/100；generic min/max 保持 VM。带 double 实际参数的整数转换、加入 int 局部拷贝等形式仍保守 VM。12 项拒绝在 auto/stage/tiled/vm 四种请求下覆盖未使用溢出局部值、显式及隐式参数转换溢出，且不为未定义 C 行为生成 oracle。该批 34.318 秒，随后原有 helper49/17 回归 37.548 秒。min 分块 Metadata3810 字节、while 形式 3655 字节；它们较普通自设分块 3739 字节略增的原因是明确参数槽与调用求值，不能为减小 Metadata 而省略转换。

四行乘八列的 AVX2 内核用八个向量累加器共享右因子，每个位置仍按递增 k 处理。128 组 FP 压力与 168 次性能对照通过。固定 $B = 64$，4 线程的两行/四行中位在 $N = 2048$ 为 32.157/30.408ms，在 4096 为 197.180/183.399ms，在 10000 为 2876.000/2527.850ms；最大规模四行区间 2514.800–2665.420ms。该批最大规模 1 线程仅一次，10.152/8.816 秒，不能当稳定中位数。较小规模有负收益，例如 $N = 512, B = 64$ 的 4 线程 1.936/2.162ms。因此自适应候选在 $N \geq 2048$ 用四行，否则两行。

独立中间规模成本批次共 200 次严格稠密对照，比较同一自适应二进制的 $B = 32/64$。1 线程在 $N = 1152$ 为 18.421/21.683ms，1536 为 42.849/52.100ms；4 线程对应 8.025/8.000ms、17.820/16.748ms。由此候选的单线程小块界限放到 1536，多个线程保留 1024；2 线程等未实测线程数属于策略外推，用户可显式覆盖物理块。本地策略不承诺在其他 CPU 或负载下最优。

八行乘四列的另一种布局将右因子载入共享给更多行，但增加广播数量。独立 128 组强制 dense FP 对照与 128 次性能执行通过，仍为负结果：$B = 64$、4 线程的自适应候选/八行内核在 $N = 2048$ 中位 30.737/31.820ms，4096 为 184.018/192.115ms，10000 三轮 2322.460/2510.260ms。

它作为编译配置实验保留，不选作默认；不能只因理论载入复用增加就宣称更快。

4.9 数据驱动带状专门化与新稠密工作负载

Netlib 的 `DPBTRF` 专门处理 SPD 带矩阵并提供带状存储接口 [11]。本项目仍接受并输出完整 N^2 布局，先检查下三角的确切非零半带宽 w，仅在 $w \le 128$ 且 $4w < N$ 时启用带状路径；显式`--structure dense` 或较小 band-limit 可以关闭它。结构由实际数据扫描确定，不由文件名、Shape 或预置标签确定。

对行 i 取 $f = \max(0, i - w)$。带内 (i, j) 仅跳过归约中 $k < f$ 的前缀，其余 $k = f, \ldots, j - 1$ 全部执行，含带内实际零。被省略前缀的一个乘数为确切零、另一个有限，初始累加器正零，在本地默认舍入环境下保持正零；之后的乘加链与原 C 相同。带外因子保持输入符号零；上三角仍为正零。不能省略中途零项，因为 FMA 下溢后的负零可能被后续加零改变。带状运算为 $O(Nw^2)$，扫描、输出分配及完整格式仍为 $O(N^2)$；当前带状 kernel 串行，指标明确记录实际 worker 为 1。

结构化里程碑 380 次原始 C 整文件对照覆盖左看、官网右看、自设分块、显式 FMA，normal/subnormal/large/tiny 下溢与符号零、奇数规模，以及带宽 127/128/129 的分派边界；46 次性能执行另外记录。稠密 Toeplitz 相关矩阵和正对角加三个外积的 low-rank 族，覆盖 2047/2048/2049 的向量策略边界，均保持各自原始 C 字节。它们未作为带状快速路径结果混入一般稠密基准。

真实 $N = 10000, w = 7$ 带状输入为 800,000,008 字节，原始官网 C 的独立编译参考实际执行 481.611 秒，输入生成 2.626 秒。完整输出 SHA256 为`fb50d92b7bb6519d52bb69afba48b36745ac56f9bf7cf3e109adbcb53154a0c2`。同批三轮旧稠密内核/带状路径 compute 中位 3002.110/184.861ms，外部 wall 中位 3877.539/1211.165ms；带状 compute 区间 172.044–215.752ms。其结构扫描 30.324–33.211ms，kernel37.618–39.199ms，读入 196.016–208.742ms、写出 598.288–655.389ms。该收益只适用于已验证带结构，不能替代一般稠密最大规模结果。

一般稠密的早退结构扫描在同批 $N = 4096$ 的 5 轮中位 226.439/233.907ms 呈负观测，$N = 2048$ 为 33.721/32.822ms；没有证据说明所有规模都因此加速。新增路径保留可关闭选项，性能波动与其他项目并发限制继续适用。

4.10 4 线程以上伸缩与真实 RAM 峰值

复用并核验已有普通稠密原 C 参考，48 次伸缩执行均保持完整字节。固定 $B = 64$、同一二进制，表4为同批轮换中位；2048/4096 各 5 轮，10000 及混合流各 3 轮。4 到 16 线程在普通最大规模约 1.56 倍，并非线性；混合流的 16 线程中位比 8 线程略慢，不能推广为任何批次更多线程必然更快。一般最大规模的 4/8/16 线程区间为 2506.240–2593.880、1704.960–1900.300、1628.130–1657.110ms；wall 中位 3549.484/3056.392/2567.551ms。IO 及分配占比随着计算缩短而增大。

通过 Windows`GetProcessMemoryInfo` 读取系统维护的 `PeakWorkingSetSize` 与 `PeakPagefileUsage`，后者是进程峰值 commit 而非实际换页量 [12]。PrivateUsage 及宿主可用物理 RAM 另以 20ms 间隔采样。普通 10000 峰值工作集约 1924.5MiB，峰值 commit 约

表 4: 伸缩批次计算中位 ms，固定 B64

工作负载	4 线程	8 线程	16 线程
普通 dense2048	38.857	27.246	25.222
普通 dense4096	194.826	132.536	120.677
普通 dense10000	2535.090	1801.070	1630.260
M3:4096/10000/4096	3380.520	2227.910	2272.450

1.879GiB；混合 M3 完整文件 1,068,435,472 字节，峰值工作集约 1924.7MiB，和单个最大矩阵相近，支持逐矩阵流式内存不会按 M 累加的结论。宿主最小可用 RAM 约 6.45GiB，但受其他项目共享负载影响，不归因于 19 号进程。资源采样会增加外部 wall 开销，不计入 native compute。

随后 81 次同一二进制的线程/几何批次比较 4 线程 B64 与 8/16 线程 B64/96/128/192。任务数量在 10000 从 B64 的 657,359 降到 B96 的 198,485、B128 的 85,320、B192 的 26,235，减少退休次数确实有潜力，但更大的串行 P/T 和缓存工作集抵消收益。8 线程 B64/96/128/192 中位 1945.690/1932.630/2334.030/2400.190ms；16 线程 2372.620/1962.650/1878.700/2289.820ms，范围大且部分重叠，和前批 16 线程 1630.260ms 也有明显共享负载波动。该批不足以单独确定默认；后续将 P/T 内独立位置向量化和高线程数 B96 组合为完整策略验证。

4.11 P/T 独立位置 SIMD 的收益与负结果

P/T 块内的每次更新都对不同输出列执行同一个已确定因子的乘加。广播该因子并向量化这些独立位置，不改变任一位置的源列顺序，也不把串行归约改成树。128 次强制 dense 浮点压力与 200 次性能执行均经原始 C 全字节校验。B64 的 2048 单线程中位 96.692/89.824ms；B96 的 4096 单线程 644.804/604.312ms，区间 632.843–657.380 与 594.514–611.965ms；其 4 线程 183.591/173.666ms，区间 177.977–194.522 与 170.364–177.117ms。这些中等规模支持 P/T 内 SIMD 的收益。

B32 的 2048 单线程 93.945/94.866ms、4 线程 36.239/39.097ms 为负观测。10000 的单线程 B64 为 8531.410/8730.380ms，仅一次；4 线程 B96 为 2835.370/2994.660ms 亦负。16 线程 B96 为 1959.040/1910.230ms，但新区间 1462–2638ms 宽，不能称为稳定最大规模收益。由此下一完整策略仅对 B 至少 64 开启 P/T SIMD，并将 8/16 线程的 4096 以上物理 B 候选设为 96；仍需组合后的轮换与 M3 流式基准决定默认。

4.12 完整线程策略验证

随后完整候选把 P/T SIMD 限制到 $B \geq 64$，并在至少 8 个 worker 且 $N \geq 4096$ 时自动选择 B96；1/4 线程原 32/64 界限保留。144 次浮点压力与 116 次严格性能执行通过，复用已核验 C 参考，整批 303.960 秒。表5比较旧 Shape 策略与完整候选的同批中位；10000 单线程仅一次，其余每配置三次。4096 的单线程 12.3% 改善主要来自 kernel 而不是增加线程，其 4 线程 14.5% 改善属于相同线程数下的内核/策略差异；16 线程不能与单线程比率相乘成官方加速比。10000 的 16 线程 compute 中位 1648.560/1368.370ms，kernel1435.880/1201.510ms，graph41.169/40.404ms，wall2625.863/2272.141ms。任务数量减少没有自动带来同等比例 graph 时间减少，因 graph 还包

表 5: 完整策略计算 ms；普通 dense 与 M3 流分别列出

工作负载	线程	旧策略	完整候选
dense2048	1	92.950	87.095
dense4096	1	715.265	627.143
dense4096	4	221.434	189.402
dense4096	8	136.119	128.316
dense4096	16	109.096	97.664
dense10000	1	9053.970	9417.710
dense10000	4	2678.300	2336.570
dense10000	8	1664.370	1700.280
dense10000	16	1648.560	1368.370
M3:4096/10000/4096	4	3282.370	3473.990
M3:4096/10000/4096	8	2578.390	2472.920
M3:4096/10000/4096	16	2505.220	2358.660

含约 400MB 因子缓存初始化。

10000 的 8 线程 compute 负观测 1664.370/1700.280ms 仍保留，即便其 kernel1492.430/1331.020ms 改善；扫描、分配与共享 RAM 压力可能影响 compute 的其他部分。4 线程新区间 2294.490–6424.710ms 包含一轮明显长尾，未剔除；旧区间 2432.280–2687.950ms。M3 的 4 线程 compute 亦负，wall 却 5943.328/5169.361ms，说明 IO/启动与 compute 口径不能混同。该批宿主最小可用 RAM 约 3.576GiB，同时其他项目使用 CPU/GPU；部分末尾观测还与下一候选构建重叠。进程自身峰值工作集 2,018,201,600 字节，约 1.880GiB，并未按 M 累加。带状 10000 两版本均用同一串行结构 kernel，其 compute200.975/226.705ms 为负观测，不应归因于未执行的 P/T 优化。此候选是可覆盖的本地成本启发式，未证明 8/16 线程在所有负载或机器上全局最优。

4.13 有界唤醒的负结果

对同一完整线程策略，实验把依赖状态更新后先解锁，再按 `min(released,workers-1)` 逐次 notify_one，完成与错误仍唤醒全部。谓词与依赖状态仍受同一 mutex 保护，208 次 FP/奇数/ISA 对照及 156 性能运行均通过原 C 字节，整批 489.754 秒。但减少被唤醒线程数没有保证总成本下降：1024 的 4 线程 7.535/9.392ms，4096 的 16 线程 104.563/113.074ms，10000 的 16 线程 1405.130/1652.730ms 均负；后者 kernel1216.810/1459.080ms 也负。

2048 的 4 线程 33.060/30.080ms、16 线程 23.362/20.842ms 呈正观测，M3 的 4 线程 3784.760/3437.740ms 及 8 线程 2616.990/2330.750ms 也正，但 16 线程 2253.080/2351.250ms 负。固定 B32 的 2048 在 4/8/16 线程 35.025/44.028/64.330ms 变成 36.377/44.746/65.189ms，均负。该候选因此保留为默认关闭的实验。每次 notify_one 本身有成本，批量释放多项时多次调用也可能比一次 notify_all 慢；这个解释是结合实现的推测，并未测量操作系统唤醒事件。下一候选仅把原有通知方式移到锁外，不改变唤醒数量，另检验因子缓存清零成本。

4.14 因子缓存初始化与原通知移到锁外

该批进一步分离两个变量：unlock 仅把原来的 notify_one/notify_all 选择移到锁外，lazy 在 unlock 基础上取消因子缓存的冗余清零。这个缓存的所有实际读取都发生在 P/T 写入之后，P 只读取本块已完成的前序位置，填充区不读取。输出 L 仍以正零初始化。另一个 poison 构建把所有缓存槽填为 NaN；poison 与 lazy 共 208 次原 C 比较覆盖 4 种源、subnormal/大数/近秩一/符号零、portable/ISA、奇数尾块、B 大于 N 和小 N 的 B1，全部通过，耗时 38.130 秒。此证据仅针对原行主序输出加因子缓存布局；后续把缓存改为累加工作区时，初始正零重新成为必要条件。

性能批次固定相同自动 B 策略，逐轮轮换三个构建，共 66 次严格核验，耗时 173.605 秒。表6保留全部分组中位；N=10000 为 3 轮，混合 M3 为 2 轮，其余亦为 3 轮。10000/4 原策略区间

表 6: 缓存初始化实验；compute 毫秒

负载/线程	原策略	unlock	lazy	lazy 相对原策略
2048/4	31.543	30.379	32.643	负
2048/16	18.694	19.922	17.206	正
4096/4	210.665	223.402	189.636	正
4096/16	106.929	110.283	100.042	正
10000/4	3196.950	2885.190	2692.640	正
10000/16	1337.160	1499.680	1485.640	负
M3/4	3748.155	3489.615	3456.605	正
M3/16	2353.970	2472.620	2056.330	正

2537.780–3271.420ms，lazy 区间 2423.210–3409.710ms，区间重叠，不声称稳定幅度；10000/16 的负观测保留。lazy 的 graph 中位在 2048/4 为 0.040ms、4096/4 为 0.136ms、10000/4 为 2.378ms，原策略对应 2.123/7.062/47.604ms，但首次触页与实际缓存写入迁入 kernel，图初始化缩短不能单独视为端到端收益。10000/16 的 kernel 原策略 1133.260ms，lazy1324.450ms，反而增加；wall 原策略 2616.967ms，lazy2464.208ms，IO 差异又改变了 wall 方向。实验时其他项目继续占用 CPU/GPU，宿主最小可用 RAM6,307,524,608 字节，进程峰值工作集 2,018,152,448 字节，均作为共享环境观测。两个开关暂仍默认关闭，不以一次建图收益替换全流程选择。

4.15 连续列块工作区的负结果与导出成本

该候选把累加区与已完成因子合并为同一份下三角 tile-major、块内 column-major 工作区。P/T 重排独立位置而不重排每位置的 k 链，乘法左、右操作数顺序仍保留。累加工作区必须以正零初始化；结束后把已完成下三角导出到完整行主序 L，上三角仍为正零。全 L 仍提前分配，因而峰值 RAM 没有降低；仅热工作集可能由原行主序 L 下三角加因子缓存约 800MB 变为约 400MB。一个 64 乘 64 块为 32KB 连续存储，原 L 的大 N 行跨度可能分散在约 64 页；这只是地址几何推断，未采集 TLB 或缓存硬件计数器。

209 项原 C 对照通过，耗时 34.512 秒；覆盖 4 种源码、FP 压力、奇数和 B 大于 N、B1、portable/ISA、1/4/16 线程，源码 block 另有 1/32/100。五项 CTest 也通过，174.410 秒。随后 94 次性能对照 168.838 秒，在表7中明确计入串行导出。除 N=10000 单线程仅 1 轮、M3 仅 2 轮

外，各配置 3 轮。不少 kernel 中位降低，但导出抵消了多线程收益；最大规模单线程 kernel 亦

表 7: 连续列块候选同批中位；全部毫秒

负载/线程	原 compute	列块 compute	原 kernel	列块 kernel	列块 export
512/1	2.768	2.625	2.193	1.884	0.141
512/4	1.718	1.585	1.133	0.880	0.149
512/16	2.125	2.207	1.558	1.443	0.157
1024/1	15.801	14.440	13.522	11.703	0.707
1024/4	6.772	6.495	4.842	3.625	0.761
1024/16	10.094	11.004	7.966	8.004	0.850
2048/1	113.152	98.758	105.140	88.019	3.497
2048/4	31.458	32.734	23.181	21.251	3.095
2048/16	19.633	23.008	11.668	10.793	3.331
4096/1	819.546	702.240	792.013	661.752	14.836
4096/4	197.625	204.194	163.918	162.591	13.906
4096/16	113.715	114.066	86.162	73.410	13.373
10000/1	9115.610	12350.500	8869.010	12055.700	84.188
10000/4	3294.160	3549.350	2965.280	3153.380	98.103
10000/16	2265.810	2271.630	2055.420	1839.410	107.687
M3/4	3748.605	4445.390	3382.410	3998.485	145.844
M3/16	2120.265	2269.390	1821.425	1874.410	124.257

明显负。10000/4 原 compute 区间 2977.950–3621.750ms，列块 3203.350–3694.340ms；10000/16 原 2101.500–2320.830ms，列块 2172.080–2562.210ms。共享宿主同时仍运行其他 CPU/GPU 项目，最小可用 RAM5,036,146,688 字节，进程峰值工作集 2,018,258,944 字节，不能用不同批次的最小值推出因果。该候选默认关闭。检查实际 AVX 反汇编，热 k 循环没有向栈 spill，故不把负结果归因为寄存器 spill。下一候选将目标累加区改为块内行主序，复用原已验证的四行乘八列内核；P/T 完成后用每 worker 小 scratch 改变为列因子布局，并独立检验最终位保持转置与并行导出。

4.16 混合布局与位保持导出的独立对照

混合候选令 U 任务的累加目标保持块内行主序，因子读取仍保持块内列主序，因而可复用已验证的原四行乘八列 SIMD 内核。P/T 拥有目标块的独占写权限，用 worker 局部 scratch 顺序形成新因子；全部累加值消耗完毕之后、依赖退休之前，把目标块转换为列因子表示。消费者仅在完成后读，因此同一工作区随任务状态改变解释，不保留两份下三角大缓存。scratch 惰性分配，每 worker 最多 $8B^2$ 字节，B96 且 16worker 理论上限 1,179,648 字节；仍提前分配完整 L，不能宣称全流程峰值 RAM 降低。

导出另有默认关闭开关：4 乘 4 AVX2 解包与 lane 置换仅搬运原 FP64 位，按独立块行分工；N 不小于 2048 且 worker 多于 1 时并行，较小 N 保持串行。奇数尾块和对角下三角独立处理，上三角不写。混合串行与快速导出共 418 次原 C 压力对照通过，51.342 秒。随后把原行主序、连续列块、混合串行、混合快速导出四个冻结构建轮换，188 次性能执行完整通过，350.395 秒。导

表 8: 混合布局批次 compute 中位；毫秒

负载/线程	原行主序	连续列块	混合串行	混合快速导出
1024/1	17.128	14.521	13.807	14.375
1024/4	6.819	7.449	6.205	6.143
1024/16	9.935	10.519	10.753	10.516
2048/1	89.991	91.815	83.187	81.595
2048/4	29.563	30.967	30.125	28.596
2048/16	20.172	20.610	22.704	19.544
4096/1	700.708	845.260	591.991	662.861
4096/4	194.568	199.421	190.820	190.838
4096/16	106.007	107.886	118.323	108.243
10000/1	9227.100	11327.100	18891.800	9791.520
10000/4	3016.970	3483.620	3039.080	2886.730
10000/16	1630.900	1999.950	1875.280	2095.940
M3/4	3451.045	3546.595	5055.220	3143.960
M3/16	1634.995	1819.310	1878.425	1963.315

出阶段的明确局部收益为：4096/4 串行 14.018ms、快速 6.350ms，4096/16 为 13.579/5.048ms，10000/4 为 95.365/33.105ms，10000/16 为 117.100/32.065ms。尽管导出减少，10000/16 compute 仍负，不能以阶段改进代替端到端收益。10000/4 原 compute 区间 2817.680–3128.430ms，快速 2870.760–3088.790ms 重叠；16 线程原 1495.190–2270.770ms，快速 1716.200–2199.220ms。M3/4 串行混合有 6660.510ms 长尾，与另一轮 3449.930ms 一起保留；表中 2 轮中位是两数平均，不称为稳定收益。

该批早段 512 出现明显共享干扰：原单线程 3.342/28.543/21.033ms，混合串行 2.644/2.574/25.616ms，快速 2.492/2.473/5.852ms。1024 原单线程亦含 129.252ms 长尾。10000 的混合串行和快速导出使用相同 kernel 代码，却单次 kernel 为 18258.800/9510.960ms，故不能把差异归因于导出开关。全部原始观察留在 JSON，未据此更新小 N 阈值。宿主最小可用 RAM5,263,216,640 字节，进程峰值工作集 2,019,373,056 字节。布局和导出开关暂仍默认关闭；下一批固定原行内核和 shape，仅检验任务图状态压缩，并记录独立系统 CPU 计数帮助判断共享环境。

4.17 当前 panel 惰性图：状态节省与性能边界

候选不再为每个 (k, m, h) 物化一个状态，而为每个下三角目标块 (m, h) 保留一个 8 字节状态，依次执行 $k = 0, \ldots, h-1$ 的 U，随后执行 P 或 T。U 结束后该状态推进一个 panel；P/T 结束后成为不可变因子。当前 U 就绪仍要求两个因子完成，当前 T 要求对角因子完成，先前 U 由状态推进隐含保证。工作线程持锁取出并复制任务，解锁仅执行计算，完成后重新持锁推进状态；就绪堆中的状态不修改。错误广播并等待所有 worker 退出，另检查无任务可执行且无 active 时的停滞，以及最终所有目标块已完成。

逻辑执行数仍为 $Q = t(t+1)(t+2)/6$，常驻状态变为 $S = t(t+1)/2$；1600 万逻辑任务保护上限继续保留。它没有省略数学工作，也没有声称输出、因子缓存或总 RAM 降为 $O(t^2)$。heap

和 bucket 分别通过 4 种实际 C 源、非融合/显式 FMA、subnormal、大数、近秩一、符号零、portable/ISA、奇数尾块与 B 大于 N，总计 418 次完整字节比较，耗时 67.036 秒。另 30 项负 pivot 测试覆盖不同 panel 及 1/4/16 线程，均在限时内退出且没有完整或 partial 输出，耗时 5.236 秒。

表 9: 同 shape 的逻辑任务与常驻状态；字节仅为任务状态和 panel 索引

N	B	Q	S	旧字节	惰性字节
1024	32	5984	528	48128	4224
2048	64	5984	528	48128	4224
4096	64	45760	2080	366592	16640
10000	64	657359	12403	5260128	99224
10000	96	198485	5565	1588720	44520

104 次性能批次复用相同原 C 真值、源码、行主序 kernel、通知与自动 B 策略，整批 209.108 秒。一般稠密 4096 的 16 线程 compute 中位旧/惰性为 102.572/95.272ms，三轮范围 99.912–109.238/94.732–99.642ms，kernel 中位 75.720/70.672ms；这个局部结果有价值，但其他配置未一致改善。10000 的 4/8/16 线程 compute 中位分别为 2844.350/2562.990、1570.820/1875.660、1628.270/1707.790ms，8 和 16 线程为负。10000 单线程仅一次 9615.620/9178.440ms，不构成稳定结论；M3 的 4/8/16 线程两轮中位为 3584.335/3743.605、2702.365/2380.530、2133.900/2127.610ms。1024 的 1/4/8/16 均为负，说明减少状态并不自动改善小任务开销。

该批 4096 第三轮 4 线程旧/惰性分别有 1957.010/1752.040ms 长尾，8 线程 977.520/988.571ms，同样保留。4 线程两次外部 wall 为 3000.990/2623.297ms，进程全线程 CPU 合计 7156.250/6734.375ms，宿主其他 CPU 时间估计 47218.750/30281.250ms。期间另一个项目进行 32700 token GPU 容量探测，主机与显存压力未隔离；这些计量只能描述共享负载，不能把尖峰归因为 DAG 实现。峰值工作集 2,018,127,872 字节，宿主最小可用 RAM5,407,789,056 字节。

4.18 当前等待计数与因子行前缀：负结果亦保留

第二候选缓存当前任务尚缺的 1 或 2 个因子通知，因子完成只减少相应当前 panel 的计数；目标推进后重新计算当前等待数，不缓存未来任意事件。每行 $\mathrm{T}(m,k)$ 依赖该行 $\mathrm{T}(m,k-1)$，最终 P 依赖最后 T，因此完成因子的 panel 形成连续前缀。以每行一个 int16 前缀替代三角索引和随机 Done 状态读取，退休时检查严格递增不变量。比较 -1 初值时使用有符号类型，避免转换成 size_t。新增目录为 $2t$ 字节，计入任务元数据；kernel、浮点表达式、自动 B 及通知策略保持不变。

heap/bucket 418 次严格原 C 对照耗时 59.524 秒，另 30 项异常生命周期 0.957 秒。156 次三候选性能耗时 605.969 秒，自动 B 与强制 B32 独立记录。表中 S 为原物化线程策略，L 为惰性 Done 检查，P 为缓存等待加行前缀；单位 ms。普通最大规模单线程仅一次，其他单矩阵三次，M3 两次。

表 10: 同批 compute 中位：状态减少未形成全局稳定收益

工作负载	线程	S	L	P
N2048	1	95.845	99.162	106.535
N2048	4	34.035	31.092	29.925
N2048	8	23.647	23.514	23.903
N2048	16	19.833	20.661	19.308
N4096	1	622.157	697.030	786.059
N4096	4	253.546	226.649	214.560
N4096	8	148.780	182.218	163.573
N4096	16	120.790	131.573	116.398
N10000	1	9415.250	9176.190	9410.580
N10000	4	3252.660	3110.070	3201.230
N10000	8	2214.760	2636.770	2789.540
N10000	16	2061.090	1785.840	1986.570
M3	4	3648.790	3545.940	3593.185
M3	8	2272.625	2788.170	2356.890
M3	16	2192.980	2604.815	3152.240
N4096/B32	4	362.082	314.891	314.839
N4096/B32	16	587.602	589.181	709.903
N10000/B32	4	4666.880	4595.630	4902.780
N10000/B32	16	10910.200	11812.000	12358.700

P 在 2048 的 4 线程局部改善，但在 4096 单线程、10000 的 8 线程及 M3 的 16 线程明显为负。M3 的 16 线程 S/L/P 进程全线程 CPU 中位为 29796.875/35679.688/42445.313ms，宿主 busy 中位 48.913%/53.757%/64.635%；其他任务未隔离，不能据此证明具体因果。强制 B32 让 10000 逻辑任务增至 5,159,805，16 线程 S/L/P 中位 10910.200/11812.000/12358.700ms，均远慢于自动 B96。P 该组三轮 11139.200–26330.500ms，最大的 26.331 秒仍与完整原 C 结果逐字节相同，不删除为异常值。

该长尾外部 wall 28428.794ms，input read 1235.830ms、graph 363.227ms、kernel 24997.700ms，进程 CPU 合计 205375.000ms（kernel 态 123234.375ms），其他 CPU 时间估计 386843.750ms。该时段 RAM 可用 7,925,399,552 字节，页错误 491,894 次；没有 hard-fault 计数，不能将其解释为磁盘分页，也不能用宿主负载独自解释全部调度差异。整批峰值工作集 2,050,871,296 字节，宿主最小可用 RAM3,449,270,272 字节。前缀模式的 10000/B64 任务元数据 99,538 字节，B96 为 44,730 字节，B32 为 393,754 字节；源 Metadata 仍为 2655 字节。$O(t^2)$ 空间改善成立，时间结果混合，两个惰性图选项继续作为默认 OFF 的研究候选。

4.19 单 worker 闭式拓扑遍历

另一默认 OFF 候选`CHOL_SERIAL_TILE_WALK` 只在实际 worker 为 1 时按目标列 col、panel $k < col$、目标行 row 执行 U，随后 P(col) 和本列 T。此前列的因子均完成，每位置仍递增 k；顺序对应 heap 的列/panel/行优先顺序。它不创建常驻 Task 或就绪队列，也不进入锁、条件变量；逻

辑 Q、初始化、实际数学任务和 1600 万逻辑 cap 保留。JSON 记录`ready_queue=closed_serial`、常驻状态与 ready peak 均为 0。多 worker 继续原物化 DAG，不声称它们有算法改动。

基础 heap/bucket418 次原 C 对照 63.918 秒；额外 224 次全部 single-worker auto/portable，覆盖四源、四 FP 模式、B1/7/31/64/96/128/1024 及奇数/尾块，62.237 秒，并断言真实 closed 遍历与零常驻状态。另 30 项错误生命周期 5.498 秒。76 次性能 306.705 秒；每单矩阵三轮，含普通 10000 及强制 B32 的单线程三轮，M3 两轮。表为旧 S/候选 W compute 中位 ms。

表 11: 单 worker 整版本对照；4 线程为未改算法的控制观测

工作负载	S 单线程	W 单线程	S 四线程	W 四线程
N512	2.735	2.599	1.793	1.862
N1024	15.805	15.421	6.485	6.797
N2048	96.382	90.506	32.563	31.145
N4096	632.523	809.327	197.345	196.032
N10000	9463.690	10204.800	2927.970	2778.400

普通 4096 的 S/W 单线程中位 632.523/809.327ms、10000 为 9463.690/10204.800ms，均为负。10000 单线程范围 S 9450.320–10210.100、W 10048.200–11316.600ms。强制 B32 的 4096 单线程 S/W 为 983.809/927.363ms，10000 为 14554.300/14533.800ms 基本持平；M3 单线程两轮中位 11330.600/11526.600ms。512、1024、2048 有局部改善，未形成普通大规模稳定收益，因此默认仍 OFF。四线程没有改变调度算法，却出现正负计时差异，说明编译实现与共享负载仍影响整版本对照，不能把所有差异归因为闭式遍历。

整批进程峰值工作集 2,050,170,880 字节，宿主最小可用 RAM6,481,620,992 字节。单 worker 常驻 Task 字节确实降为 0，完整输入、输出和因子存储仍保持，未宣称总 RAM 消除。

4.20 输入所有权复用：约 800MB 实测峰值节省

在列块统一工作区、或行累加块转列因子的混合模式，P/T/U 只读 A 并读写 packed，不访问完整 L。已有实现提前分配 L，虽缓存仅保留下三角，峰值仍为 A+L+packed。默认 OFF 的`CHOL_REUSE_INPUT_BUFFER` 提供`execute_owned(Matrixinput,...)` 入口；CLI 将读入的 vector 移动到该入口，不复制 payload。只读`execute(constMatrix\&,...)` 保持输入不变，不能复用输入。所有者输入的释放归 execute_owned，在成功返回或异常展开时完成；旧 CLI 变量原先直到写出结果后才释放，故新旧销毁计时不可直接混同。

第一阶段只对显式`--structuredense`、已证明 native 图且列块布局生效的 owned 入口启用。先只分配 packed，完整 output vector 保持空。`program.run` 等待并 join 全部 worker，验证完成数之后，输入再无读取；最后导出只读 packed 及构造时缓存的 n/B。此时把 A 的 vector 交换给 L，逐行将上三角填为正零，再覆盖每个实际下三角位置。奇数与尾块仍截断真实维度，padded 不会导出；数学和每位置 k 链不变。generic VM、stage、const API 及结构路径继续旧分配。异常在交换前发生会释放 owned A/packed，并丢弃整个 M 的 partial。

同源 control 与 candidate 均为 hybrid+ 快速导出 + 向量扫描，且均走 owned API、输入销毁包含在 compute；唯一编译开关差异为复用开关。418 次四源 FP/边界原 C 比较，两版分别

209 次，并断言每矩阵复用计数正确；另 100 次 M4 混合维度、各 backend、generic 读上三角与不同 simd 对照，18 项先成功一矩阵后 late-pivot/NaN/truncated 失败。原 A 上三角包含巨大有限值、负值和负零，Cholesky 最终上三角仍全部正零；generic 能正确读取全部原 A。API 单测另外验证 const 输入不改、owned 返回 vector 的 allocation 身份、原 C 整 bytes。18 项失败进程另记录全程 peak WS/commit/CPU 和宿主计量，最大工作集 58,961,920 字节；这些小失败 Shape 不代表失败 N10000 峰值。每项 30 秒退出界限，均无最终文件或 partial 残留。

80 次资源/吞吐对照耗时 201.130 秒，不重算昂贵最大 C oracle；所有输出整文件严格相同。下表 compute 三轮中位为 ms，WS 为该配置三轮最大工作集 MiB，S/R 表示同源不复用/复用。

表 12: 输入复用：吞吐和实际工作集独立记录

N	worker	S compute	R compute	S WS	R WS
1024	1	13.528	12.444	25.6	17.6
1024	4	6.399	5.435	25.8	17.8
1024	16	10.671	9.310	26.3	18.3
2048	1	84.188	79.440	86.0	54.0
2048	4	29.370	25.657	86.2	54.2
2048	16	20.084	16.919	86.9	54.9
4096	1	672.677	599.772	326.8	198.8
4096	4	188.434	165.486	327.1	199.1
4096	16	112.035	103.354	329.6	201.6
10000	1	9824.830	9099.830	1924.1	1161.2
10000	4	3295.490	2930.670	1924.4	1161.4
10000	16	2152.930	2036.520	1925.8	1162.8

10000 的 4worker 峰值工作集 2,017,865,728/1,217,859,584 字节，commit2,017,062,912/1,215,492,096 字节，符合省掉约 800,000,000 字节完整 L。M3=[4096,10000,4096] 的 16worker 峰值 2,019,360,768/1,219,375,104 字节，不按 M 倍增。其 4worker compute 两轮中位 3866.090/3639.475ms，16worker2495.990/2260.175ms。全批宿主最小可用 RAM6,322,229,248 字节；资源节省是直接测量，不需要将所有时间差归为缓存或分页。

增加 upper 清零令 10000 的 4worker export 中位 33.506/57.043ms、16worker29.998/59.780ms，且这些已计入 compute。依然保留单轮吞吐负结果：4worker 第 0 轮 2810.190/2930.670ms，16worker 第 2 轮 1897.410/2385.770ms；4096 的 16worker kernel 中位 74.026/78.291ms 亦负。它减少整 L 初始分配/清零，不能据此保证每个 kernel 更快。这是相同 hybrid 布局内部的所有权单开关比较，不是相对历史 row 布局的单变量加速。

证据`artifacts/input-reuse-milestone/input-reuse-{stress,performance}-report.json`、`artifacts/input-reuse-lifecycle-extra/test-report.json` 及`artifacts/input-reuse-failure-probe/test-report.json` 含每个二进制/source hash、资源与失败。研究开关仍默认 OFF；后续独立阶段才扩展实际结构分类失败后的 auto 输入复用。

4.21 auto 延迟分配与最终双配置选择

auto 扩展把结构证明放在完整 L 分配之前。实际带状/对角证明成功，才为结构 kernel 分配完整正零 L，owned A 仍只读；结构证明失败时，只有获完整 native 图证明的列块工作区可继续保持 L 为空，全部任务读取结束后复用 A。generic、stage 及 const API 仍保持旧分配。即使请求 tiled，结构成功也优先用已验证的 band/diagonal；memory 配置不声称结构路径峰值也降到 800MB。

同一当前源分别编译 row profile 和 memory profile：二者 owned API、扫描、B/线程策略一致；memory 另外开启混合布局、转置导出及输入复用。因此这是整配置比较，不隔离所有权和布局的单一因果。418 次 heap/bucket 四源 FP 压力、336 次结构与新 dense 补充、100 次 backend/M4/generic 原 C 比较，共 854 次完整文件比较通过。另 288 非有限拒绝、18 多矩阵失败及 24 项含 late-band-pivot 的资源采样失败，均无输出或 partial 残留。

两个测试脚本问题另存证据。结构压力中曾误将 M 最后矩阵的`selected_engine` 当所有矩阵的分派，错误要求混合 M 累计复用为零；小 N 可合法 dense 回退，runtime 与原 C 正确。修正为 aggregate 计数范围、control 为零、所有对角 M 为零，保留失败 checkpoint，并不重跑已完成 418 项。性能脚本 M 拼接临时变量遮蔽报告 Path，导致 JSON 保存至临时 M oracle；长期原 C 参考未修改，M 阶段尚未开始。主动停止自身 benchmark，保存 39 次已逐字节通过的普通 dense 记录，修复变量并重建 M 参考，SHA 与先前独立原 C 拼接 manifest 一致；再完成 35 次。最终 74 次通过，322.990 秒是两段活跃耗时之和，不含中断间隙，未丢弃此前负长尾，也未重算昂贵 C oracle。

下表三轮 compute 中位 ms，WS 该组三轮最高 MiB；R/M 为 row/memory。同源数据、原 C 真值与每次 CPU/宿主量均保存。

表 13: auto 整配置对照：明显吞吐负结果亦保留

N	worker	R compute	M compute	R WS	M WS
2048	1	88.822	78.964	86.0	54.0
2048	4	33.476	27.767	86.1	54.2
2048	16	20.781	19.968	86.4	54.9
4096	1	6752.640	627.413	326.8	198.8
4096	4	224.084	1453.780	326.9	199.1
4096	16	120.411	115.141	327.2	199.7
10000	1	9194.470	12984.300	1924.1	1161.2
10000	4	3071.660	2231.550	1924.3	1161.4
10000	16	1729.450	1526.820	1924.5	1162.1

10000 的 4worker 峰值 WS2,017,746,944/1,217,859,584 字节，commit2,016,944,128/1,215,504,384 字节；M3 的 16worker 峰值 2,018,160,640/1,218,924,544 字节。带状 10000 成功路径两者仍约 1,606MB，auto compute176.412/228.659ms 为负，portable204.076/202.043ms 接近。M3 的 4worker 两轮 compute 中位 3256.665/7129.730ms 明显负，范围 2851.800–3661.530/

3499.860–10759.600ms；16worker2221.750/1789.895ms，外部 wall3366.740/3323.664ms 基本接近。不能由内存节省保证吞吐改善。

10000 row 的 4worker 第 2 轮 18680.600ms，进程 CPU68484.375ms、宿主忙 55.504%、其他 CPU 估计 297203.125ms，可用 RAM7,041,765,376 字节，页错误 493781 次。memory 单 worker 第 1 轮 16008.300ms，CPU15093.750ms、其他 CPU162437.500ms；M3 memory 的 4worker 第 1 轮 10759.600ms，CPU40046.875ms、其他 CPU200750.000ms。4096 row 单 worker 两轮 6752.640/6783.120ms 同样保留。全批最小可用 RAM6,659,747,840 字节；没有 hard-fault/温度/频率计量，不把这些长尾归因于分页或某个单一 DAG 改变。

最终选择保守 row 作为`local-mingw` 默认，另提供显式`memory-mingw`，输出目录`build-memory`。两者 width16、rowgroup0、CPU 检测 SIMD、P/T 向量化、向量扫描、选择通知开启，packed 初始化正零；惰性图、等待前缀、闭式单线程、有界/锁外通知关闭。memory 额外开启 COLUMN/ROW/TRANSPOSE/REUSE 四个选项。普通构建选项默认仍 OFF，通过 preset 显式控制，可按资源目标选择；不以混合局部结果强行宣称全局更快。最终 native 优化冻结后仅执行两配置验证和报告定稿。

4.22 最终构建与冻结验收

两个正式 preset 已分别重建，native 代码停止优化。`local-mingw` 的 7 项 CTest 全部通过，159.28 秒；`memory-mingw` 亦 7/7，199.42 秒。每配置 full 基础流水线 13 种源、188 次整文件原 C 比较、24 项拒绝；helper49 次/17 拒绝，while63 次，整数选择 helper105 次/12 拒绝。因此每配置四条源码流水线共 405 次原 C 比较、53 项拒绝；另有原生所有权、已知矩阵/二进制格式和 308880 次扫描位/保护页单测。两个构建的 artifacts 以 build 标签分开，CTest 日志、结果 SHA 与对应 binary 关联保留。

最终二进制另外各做普通 dense10000、真实 band10000/$w = 7$、M3 三项，共 6 次整文件严格验证，46.277 秒；只执行各一次，不能据其时间给稳定性能结论。默认/memory compute 分别为 2686.230/2454.770ms（dense）、187.096/1041.010ms（band，明显负）、3133.030/2628.060ms（M3）。dense 峰值 WS2,017,636,352/1,217,859,584 字节，M3 峰值 2,017,918,976/1,218,039,808 字节；band 两者约 1.606GB 且复用计数为 0、实际 worker1。普通单矩阵 memory 复用 1 次，M3 复用 3 次，default 全部 0。严格整文件 SHA 匹配既有原始 C 参考，结构和多矩阵边界未以小测试替代。

`artifacts/final-profiles/build-provenance.json` 记录所有源码文件 SHA、canonical source manifest SHA、preset、两版二进制和编译 flags/link/cache、四条流水线与 CTest 日志，`scale-report.json` 记录最后 6 次资源/时间/真值。preset SHA256 为`479a0334726ca1d99d10e4f536073886e631e66174270eec564effa9ed1c571d`；源 manifest 为`f9a89e30932665402320ca7126561e07f9154a28f79ccebafd0912d32784ca94`，定义为各 source 相对路径到 SHA 字典的 sort_keys、紧凑 JSON UTF8 编码 SHA。最终源码/编译配置快照独立保存，报告自身另有冻结 SHA，不纳入此构建 source manifest。

4.23 历史整版本对照的实现限界

历史 S 基线为冻结 ca0 版本。混合块布局为通用`ordered_update_avx2` 增加`logical_size` 参数，使块内 stride 为 B 时仍按实际 N 决定两行或四行分派。布局关闭后数学内核、归约及操作数顺序不变，但此 ABI 改动仍进入后续惰性图和单线程遍历构建，机器码与历史 S 不能假设完全相同。因此相关表格是整候选版本的同批比较，不单独证明布局、调度或计数器的因果贡献。源与构建快照保留此差异；后续扫描实验从相同当前源码、相同计时字段分别构建标量 control 与向量 candidate，进一步减少实现混杂。所有历史逐位正确性证据仍然有效。

4.24 同源有限值与结构扫描：局部收益和负结果

`CHOL_VECTOR_SCANS` 从同一当前 runtime/math/ABI 及计时字段构建标量 control 和向量 candidate，减少历史构建混杂。整数位操作检查 FP64 指数是否全 1，覆盖完整输入，含不用的上三角；每次至多读取 4 元素，尾部逐元素。有限值检查之后，结构扫描取每行下三角第一个非零元素，其距离等于该行最大半带宽。绝对值位掩码令 $+0/-0$ 均为零，subnormal 仍为非零。AVX2 需 CPU 检测且`--simdauto`，portable 仍有标量实现，数学链与融合语义不变。intrinsic 位操作语义见 Clang 头文件文档 [13]。

共同压力 209 次、补充带宽/浮点/新稠密族 336 次，总计 545 次独立原 C 整文件比较通过。另 288 项非有限拒绝探测 ±Inf、quiet/signaling NaN 在首末、向量边界和上三角位置，均不发布输出或 partial。308,880 次单测与标准库标量比较，长度 0–65、偏移 0–7、符号零、subnormal、最大有限值及非有限位模式；Windows 不可读保护页尾长 0–33 未越界。quick CTest 两项通过。共同压力 24.626 秒，补充批含共同阶段 76.988 秒。整数位检查不对 sNaN 执行浮点算术；测试保证值域和拒绝行为，不承诺浮点异常标志相同。

60 次交替顺序性能耗时 174.734 秒，复用验证过的原 C 真值，每次整文件核对。表中 S/V 为同源标量/向量，单位 ms，均三轮中位；validation 是 compute 的子集。

表 14: 同源扫描对照，一般稠密负结果保留

N	worker	S compute	V compute	S validation	V validation
1024	1	15.998	17.065	0.308	0.238
1024	4	6.873	6.761	0.504	0.284
2048	1	109.942	151.030	1.586	1.598
2048	4	36.243	34.047	1.405	1.333
4096	1	820.546	823.971	6.591	5.172
4096	4	188.954	197.126	5.343	4.646
10000	1	9482.680	9570.510	31.473	26.005
10000	4	2961.660	2639.460	32.457	24.183

普通 10000 的 4worker compute 为 2961.660/2639.460ms，但单 worker9482.680/9570.510ms 负。2048 单 worker109.942/151.030ms 负，而 validation1.586/1.598ms 相近，不能归因于扫描

kernel 的时间差；4096 的 4worker188.954/197.126ms 也负。输入校验几十毫秒，相对稠密 $O(N^3)$ 数学量小，不能宣称一般稠密全局加速。

真实 10000、$w = 7$ 带状 auto compute 中位 167.145/147.255ms，validation31.745/24.879ms，结构扫描 30.885/17.077ms，外部 wall1082.146/1021.630ms。请求 4 线程而实际带状数学仍为 1worker，JSON 分别记录。portable compute177.979/174.942ms 近持平，结构 31.114/30.313ms。完整 800MB 输入/输出的 I/O 与分配仍存在，不能把扫描阶段收益当端到端同比收益。整批最高工作集 2,017,734,656 字节；共享 GPU/CPU 负载未隔离。

`artifacts/vector-scans-milestone/scan-stress-report.json` 和`scan-performance-report.json` 记录补充压力、所有负值与计量，共同压力另保留`vector-scans-stress-report.json`。二进制与编译源/flags 分别冻结。最终两个 preset 均显式打开扫描开关，依据有限值/结构扫描局部收益，仍保留普通 dense 负观测；普通 CMake 开关默认 OFF。

5 工作负载、硬件与测试时间成本

5.1 本机环境和能力需求

本机为 AMD Ryzen 9 8945HX，16 物理核、32 逻辑处理器；RAM 为 33,605,316,608 字节，约 31.30GiB。系统查询的 L2 总量为 16MiB、L3 总量 64MiB；未设置核亲和性或隔离其他项目负载。开发环境为 Windows、GCC15.2、Clang21.1、CMake4.2 与 Python3.12 标准库 venv。现有执行器使用 CPU，不要求 GPU 或 NPU。

软件能力包括 C/C++ 编译、AST/IR 与类型分析、符号循环域及依赖证明、线程同步、缓存布局、IEEE 浮点语义、二进制流式 IO 和独立参考测试。硬件能力要求可靠 FP64、足够 RAM 与临时磁盘；AVX2/FMA 为检测后使用的加速路径，基础 CPU 路径保留。常规研究用 4 工作线程，资源伸缩研究另测 8/16 线程；其他项目仍可能同时开发。

表 15: 一般稠密单矩阵工作量与数据规模；十进制 MB/GB

N	约 $N^3/3$ 浮点操作	单文件 MB	$A + L$ MB	打包缓存约 MB
512	44.7M	2.097	4.194	1.049
1024	357.9M	8.389	16.777	4.194
2048	2.863G	33.554	67.109	16.777
4096	22.906G	134.218	268.435	67.109
10000	333.333G	800.000	1600.000	400.000

公式忽略 $O(N^2)$ 除法、开方及尾块；不是硬件执行指令计数。$N = 10000, B = 32$ 的真实打包分配为 402,563,072 字节，输入、输出和打包合计约 2.003GB，再加任务元数据、队列、线程栈与运行环境。单矩阵的输入、参考输出、待比较输出约占 2.4GB 临时磁盘；流式处理使 M 影响累计时间和文件大小，而不是把全部矩阵一起驻留内存。研究参考当前保留以复用原 C 真值，临时混合流在核验后删除。普通/带状两组最大参考及其他里程碑已占约 4.7GB，完整研究含混合 M3 临时输入/参考/输出宜预留至少 8GB；最终清理后可只留 hash 与时间证据。

5.2 计时口径

最新图批次使用 GetProcessTimes 取得进程 kernel 与 user CPU 时间，合计所有线程，因此可大于外部 wall；GetSystemTimes 在本机 32 逻辑 CPU 上累加宿主时间，kernel 含 idle，计算 busy 时显式减 idle[14, 15]。宿主 busy 减进程 CPU 仅作为其他任务的近似背景，包含 IO、启动及采样程序，不能分离 kernel 内竞争。FILETIME 的单位是 100ns，本机短测实际出现 15.625ms 量化，不把单位当精度。PageFaultCount 包含所有页错误，不是 hard fault 或磁盘分页计数；没有性能计数器证据时不声称缓存/TLB 或分页成因。`compute_ms` 在 Runtime 内部围绕 `execute` 计时，当前 CLI 改用`execute_owned` 并把输入释放计入 compute；历史 const 入口输入通常直到写出后释放，二者不直接横比。包含输入有限值扫描、输出与打包分配、图构造、调度、线程创建/退出、kernel 及工作区导出；不含文件读写。`graph_ms` 包含任务与打包缓存初始化，`kernel_ms` 包含工作线程运行与依赖队列，`export_ms` 单独记录工作区到完整 L 的导出，三者仍不覆盖所有 compute 开销。`input_validation_ms` 单列完整输入有限值扫描，计入 compute；结构扫描亦另列。`input_read_ms`/`output_write_ms` 包含对应文件操作与末尾检查/刷新，Metadata 读取单独计时。外部 `wall_ms` 还包含进程启动、动态库加载等；不能直接把 wall 减 compute 称为纯 IO。

每次性能运行之后的全文件字节比较及 SHA 计算不包含在 Runtime compute 或 wall 中，完整里程碑耗时包含参考编译、数据生成、C oracle、全部执行与比较。第一批采用 5 轮轮换顺序，第二批较大规模采用 3 轮或最大规模 1 轮；其他项目可能并发，未固定频率、冷热缓存或温度。重复读取通常使用暖文件缓存。由此可以观察局部优化收益，但不能把它们作为官方机器上的稳定得分预测。

5.3 结构化最大 Shape 与一般稠密最大 Shape

早期 $N = 10000$ 测试使用完整 800MB 二进制布局的对角 *SPD*：运行时先扫描数据确认严格对角结构，再开方并保留符号零。compute 为 394.937ms，外部 wall 约 6.735 秒。它证明格式、最大 Shape 与数据驱动专门化，工作复杂度近 $O(N^2)$ 扫描；一般稠密 Cholesky 约 $O(N^3)$，不可用其时间替代。

本轮一般稠密 $N = 10000$ 已实际执行约 333GF 工作、原始 C oracle 及完整结果比对，约 13.7 秒的 4 线程计算也来自稠密数据。第二批整体耗时 1023.768 秒，主要成本包括 504.670 秒原始 C 最大规模参考、34.767 秒数据生成和多次较大规模运行。后续性能搜索可复用不变源码和数据的已核验参考文件，最后仍进行完整字节核验，避免重复支付参考计算成本。

6 本地语义限界与后续工作余量

当前能力覆盖明确 C 子集与特定可证明 Cholesky 运算图，不能声称任意 C 自动并行。指针别名、二维 VLA、堆分配、一般调用图、可变 helper 与控制流 helper 尚未支持。通用 VM 串行、受指令限额约束；有符号 int32 的未定义算术拒绝。Metadata 是本项目受信编译器与 Runtime 之间的契约，Runtime 不会重新从审计表达式 DAG 完成通用依赖证明，官方 Metadata 接口若另有规定仍需适配。

下一阶段价值较高的工作包括：将已验证的 $O(t^2)$ 当前 panel 图扩展为一般符号依赖实例化；

分散就绪队列和工作窃取以降低中心锁与优先堆成本；限制 lookahead 并回收已完成 panel 缓存；根据真实规模、线程数、ISA 和成本模型选择物理 B；证明更多合法循环和存储形式。上述改动均必须维持每位置正向归约链和独立原始 C oracle。GPU、分布式或 BLAS 替换还需要额外浮点契约与测试，未作为已完成能力。

前端覆盖与性能优化都有剩余空间，但本地初步目标已经由实际 source 到动态 Shape、任务 DAG、严格参考结果的端到端证据支撑。正式提交前仍需对齐官方 helper、评测环境、Metadata 评分口径和隐藏数据；没有官方材料的部分维持显式本地假设。

表 16: 后续工作余量：单人估算，非实测工期或交付承诺

方向	工作日	对应验证时间估算	设备与临时资源
前端 helper/IR	5–10	更多控制流、别名、存储形式：每批 5–15 分钟；含新最大规模 C oracle 另加约 8–10 分钟	现有 CPU；约 4GB 临时文件
稀疏专门化	4–8	不规则稀疏依赖与浮点证明：每批 10–30 分钟；保留带内零、FMA 与完整稠密负例	现有 CPU/RAM；6–10GB 磁盘
Shape 成本策略	2–4	规模、1/2/4/8 线程和 ISA 交叉轮换：每批 15–40 分钟；温度/亲和性控制另计	现有 CPU；无需 GPU，约 4GB 缓存参考
官方集成	2–5	获得契约后格式/Metadata/机器适配：完整批次 20–60 分钟；隐藏评测等待不计人工	官方机器能力尚未知；本机参考约 4–6GB

上述时间为基于已测本机成本的估算，不代表官方环境。已有实测为最新 49 对照加 17 拒绝 helper 批次 27.294 秒、计数型 while 批次 20.073 秒、结构化批次 258.358 秒，最大带状原始 C 参考另用 481.611 秒，一般稠密最大参考 504.670 秒。引用核验不变参考文件可节省重复 C oracle 成本；源码、输入或 hash 改变时须重新生成。

A 证据与复现

核心文件为 `tools/c_frontend.py`、`semantic_cholesky.py`、`analyze_source.py`、`src/runtime.cpp`、`tiled_runtime.cpp` 与 `simd_kernel.cpp`。证据位于各里程碑 JSON，报告中的数值已内嵌，阅读与编译无需访问外部数据文件。

```
.\scripts\bootstrap.ps1
.\scripts\test.ps1 -Profile full -LargeShape 10000
.venv\Scripts\python.exe tools\benchmark_tiles.py
.venv\Scripts\python.exe tools\benchmark_extended.py
.venv\Scripts\python.exe tools\benchmark_microkernels.py
.venv\Scripts\python.exe tools\benchmark_simd.py
```

历史 A/B 工具需要保留对应基线二进制；其 SHA 是审计指纹，不用于计算分派。最大规模参考文件在最终核验后清理，复现长批次将重新生成数据并编译原始 C 参考。

本报告各性能批次使用同一官网转录 C 源码，SHA256 为0363f60d7f848b33c4e42a2be0d15d8d8d89c87caeae6b103f6a3d2059fd3a39；左看、自设分块、显式 FMA 另用于正确性压力而不混入普通 dense 性能表。各保存的源/metadata 审计及大参考 manifest 可交叉核查来源；较新的性能 JSON 直接内嵌 source hash。

表 17: 历史和最终二进制 SHA256

版本	SHA256
阶段 1 旧列线程池	fc69cc2f829421e4ebc022fff317a3aac6bdca0cfa00307e4b406af501701cc4
紧凑图 4 路	964b8eccdf4d95f39dfec0609587023f4c1d74c88d49e4d003a08cdeed059931
候选 8 路	8520e955958f23731a850088e3969cf776570b35364999956da64973853b7607
候选 16 路	74625dbce703bed149e4e205eeb9b30cd5a7ba61ec957ba53d45574155ed4446
初代块 DAG	213fbecd3faf5ce897e71cb76a547f0b8ecc59d665a319c362933ec29888053e
检测 AVX2	27057f3a7450cf52abf117f1ed0074294a12854324e1c78ec43b2b976c8a2609
P/T 因子缓存	e7594cdca7a63437e753a3b7a3d9aa5e267957d04caaeb35d36c53b41d699e6f
64 字节对齐与 heap	665f6dc687a70cad07486c881180704897c85667ca7b9a99c3f9c9ae2b76410a
两行 AVX2	cd4b421105fef36948efbb968b1b4f0c5899aec235dce2177ec4b9514266a72e
两行加选择通知	e1e6ec1bdb2de90e1e5396fc541dd50509e4c389b06150d23f13799bc249031f
自动物理块候选	03aee171a3cad76d0cfba0e6879e3406567142795cddb420981c8f94824cb6f3
四行 AVX2	a69afe8a378c78c7f3e8f984edaf7b5360cac30c5d923d7f484ba37f272db954
两/四行自适应	59ff609e2c36ede9204f24ff4104680369fe5d349b3ca2ef267dac3b47123327
带状路径候选	fc511dda36fac260ef043aa4cf5a609d18c54e89a6363717b37eb97390ef2025
线程相关 B 候选	378179fe3efbffbb6f6d581e412a2e93613e6040acda362bcbfbd202c12ad997
实验八行 AVX2	2757dd1ac400ce39aaeaa55159080129d05140728be9b8e7a710bb2013d772b7
P/T SIMD 未限 B	06f92cb061165868a1841182629afc134faacbcd210ff73467e4319c574bde88
完整线程策略	ca0b49fce8863690bb6cb72ffd31de1110ef2e0bb28d4e0722c95a66693fce1a
有界解锁通知	89baaba88fc8aaf2aed4a7444e1d1c0a536ad4a591ab3a9392efa74003a983ec
原通知移到锁外	25a036f5606edad86d67977f51952612ff6f4dd6440104d9c2a4a5222f3f5b10
NaN 缓存初始化	2c790e79a5ada84f76ff0c7a265282f9e3daf06c20afe5677ed181c6d062099f
取消缓存清零	2c2e637e24e14faf8fabe125e517ee065e5d62ee9f1a6334797671b120478d0b
连续列块工作区	ca2feb08907f2c346cccff1e6e7193f3200b6c39389cdbad0c48e8fefcad956a
混合块布局串行导出	d721b9c855120b27c63bd7d2ee5a0a79e0bc7d75cdba343542775ed28e599fe9
混合块布局快速导出	496adfce08ac1bee8bcb79522ef9d93e74fc6cdfb2c66e7c0976993068c3b25e
当前 panel 惰性图	44ffaf2780f649edccf33c02ea82601ee04a16b351fa53f446d38f76afc60936

版本	SHA256
当前等待与行前缀	`566326c9e80de621ec521485f3d2bcb776f28fd3064f296f00248375f18e290b`
单 worker 闭式遍历	`e81fa039c23103aefcfd954a96764c7aec2c9dfb38a88246cb67d8ff6e7d780b`
向量扫描标量 control	`daa2b283b887d9d47ca61565afb5074072f904fd8c051d6b134505fea7a1e837`
向量扫描 candidate	`9adb63c919737941271df4b1fefb62414e1967ace2d0cd93254c831c4b669fc8`
owned 同源不复用 control	`79ab041bc94bf4663b515819075a74511c3265d1fc1bc2dce2e68cd09fc09d54`
owned dense 复用 candidate	`6a9d3ecbf578524f3e0950f4b41e99c911de2fae10ceca46d3e75cbc86b020a6`
auto 同源 row profile	`b793c1ee2db42efd601e50233701716214e6aed3891e9df535b68978cfb05415`
auto memory profile	`6ee0c07b40e60bd9b38d82e8d0c8fbb4cbbd4363a4b688f182b138639b4758e6`
最终 local-mingw 默认	`2b05a6db918ced0c9c9e3bb1bfe5159ea5ed3cfee3ccb0633a538f12de409e62`
最终 memory-mingw 显式	`7da06bbd96b711f08283c1df2379741f70d8fb2be6727feae586f6c00aaad947`

参考文献

[1] Chaspark，2026 Hackathon 赛题 19 详情，2026-10-10 复核。https://www.chaspark.com/#/races/competitions/1326691788365586432。

[2] LLVM Polly, Architecture. https://polly.llvm.org/docs/Architecture.html。

[3] A. Buttari, J. Langou, J. Kurzak, J. Dongarra. A class of parallel tiled linear algebra algorithms for multicore architectures. *Parallel Computing*, 35 (2009), 38–53. DOI:10.1016/j.parco.2008.10.002. 作者机构全文：https://icl.utk.edu/files/publications/2010/icl-utk-440-2010.pdf。

[4] Netlib LAPACK, DPOTRF documentation. https://netlib.org/lapack/explore-html/d2/d09/group__potrf_ga84e90859b02139934b166e579dd211d4.html。

[5] University of Tennessee ICL, PaRSEC Overview. https://icl.utk.edu/parsec/overview/。

[6] Inria, StarPU tutorial, 2014 PATC. https://starpu.gitlabpages.inria.fr/tutorials/2014-05-PATC/。

[7] LLVM, Language Reference: Floating-Point Environment and Fast-Math Flags. https://www.llvm.org/docs/LangRef.html。

[8] Clang, Users Manual: Floating Point Options. https://clang.llvm.org/docs/UsersManual.html。

[9] P. Ahrens, J. Demmel, H. D. Nguyen. Algorithms for Efficient Reproducible Floating Point Summation. *ACM Transactions on Mathematical Software*, 46(3), Article 22, 2020. DOI:10.1145/3389360. 作者公开全文：https://people.eecs.berkeley.edu/~demmel/ma221_Fall23/J115_Efficient_Reproducible_Summation_TOMS_2020.pdf。

[10] GNU Compiler Collection, x86 Built-in Functions. https://gcc.gnu.org/onlinedocs/gcc/x86-Built-in-Functions.html。

[11] Netlib LAPACK, DPBTRF documentation. https://www.netlib.org/lapack/explore-html/dc/d58/group__pbtrf_gafd2c3ba375ecee6dba1fba62696993e0.html。

[12] Microsoft Learn, PROCESS_MEMORY_COUNTERS_EX. https://learn.microsoft.com/en-us/windows/win32/api/psapi/ns-psapi-process_memory_counters_ex。

[13] Clang. AVX2 intrinsic header documentation. https://clang.llvm.org/doxygen/avx2intrin_8h.html. 在线版本不等于本机实际 GCC 15.2/Clang 21.1。

[14] Microsoft Learn, GetSystemTimes. https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-getsystemtimes。

[15] Microsoft Learn, GetProcessTimes. https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-getprocesstimes。